Universidade Federal do Rio Grande do Sul – Instituto de Informática
INF01203 – Estruturas de Dados
Professor Dennis Giovani Balreira
2026/2 – Turma C

Especificação do Trabalho Prático Final

1. Objetivo

Este trabalho visa proporcionar aos alunos a oportunidade de integrar, em um sistema real, as estruturas de dados estudadas ao longo da disciplina. O trabalho exige não apenas o **uso correto** das estruturas, mas também a **medição e análise comparativa do desempenho** de operações-chave, relacionando os resultados à teoria vista em aula. A nota corresponde a 15% da nota final da disciplina.

2. Reaproveitamento de código

As implementações das estruturas de dados desenvolvidas ao longo da disciplina foram dadas em aula e **devem ser usadas como ponto de partida** — o objetivo *não* é reimplementá-las do zero. O grupo **deve** adaptá-las e organizá-las conforme julgar adequado: ajustar a interface de funções, adicionar campos às estruturas, reorganizar módulos, renomear identificadores, compor TADs entre si, etc. A qualidade dessas decisões de projeto é um **critério de avaliação**; qualquer modificação relevante deve ser descrita e justificada no relatório.

O trabalho pode ser desenvolvido em **linguagem C ou C++**. **Não** é permitido utilizar estruturas de dados prontas de bibliotecas externas ou da linguagem padrão (como `std::map`, `std::priority_queue`, `std::unordered_map`, `std::stack`, `std::queue` ou similares). Todas as estruturas devem ser as da disciplina, vistas em aula, podendo ser adaptadas.

3. O jogo

O trabalho deverá ser realizado em **duplas (fortemente recomendado)** ou individualmente.

O jogador é o mais novo funcionário do ***Dado Burger*** (Figura 1) e deverá atender aos pedidos de uma série de clientes ao longo do dia.



Figura 1: Ilustração do jogo Dado Burger.

3.1. Menu e histórico

A tela inicial deve ser um menu com três opções: **Novo jogo**, **Como jogar** e **Histórico**.

A opção **Como jogar** exibe uma tela estática com as regras básicas: o objetivo do jogo, a lista de ingredientes e seus símbolos (conforme Tabela 1), as ações disponíveis e o que cada uma consome de turno, e as condições de vitória e derrota. Não há interação além de retornar ao menu.

O histórico registra em arquivo o nome e a pontuação de cada jogo encerrado. Ao abrir o histórico, o usuário pode ordenar por **ordem alfabética** ou por **pontuação decrescente**. O código de **um algoritmo de ordenação eficiente** foi dado em aula; o grupo escolhe qual usar e deve medir seu tempo de execução. A escolha deve ser justificada no relatório.

3.2. Parâmetros configuráveis

Os valores abaixo ficam a critério do grupo e devem ser documentados e justificados no relatório. Não há resposta certa: o grupo deve equilibrar jogabilidade e desafio.

Paciência máxima do cliente (turnos): deve tornar o jogo desafiador mas jogável.

Número de clientes por partida: determina a duração da sessão.

Validade dos ingredientes perecíveis (turnos): pode ser diferente por ingrediente.

Tamanho da tabela hash (m) e função de hash: justificar em função do número esperado de pedidos.

Qual algoritmo de ordenação utilizar: dentre os das aulas.

3.3. Turnos e clientes

O jogo é baseado em **turnos**: cada ação do jogador consome um turno.

Pedidos. Para cada cliente, um código de pedido é sorteado aleatoriamente dentro do intervalo de pedidos disponíveis na AVL e buscado de maneira eficiente. Este é o pedido que o jogador deverá atender para aquele cliente.

Fila de clientes. Todos os clientes da partida são enfileirados no início do jogo, na ordem de chegada. O jogador sempre atende o cliente na frente da fila. Ao concluir o pedido desse cliente, ele é removido da fila e o próximo passa à frente.

Paciência. Cada cliente tem uma **paciência máxima** (em turnos, *definida pelo grupo*). A paciência do cliente da frente da fila é decrementada **após cada ação do jogador** que consome um turno. A paciência dos demais clientes na fila **não** é decrementada enquanto aguardam.

Derrota. Se a paciência do cliente da frente chegar a zero antes de o pedido ser concluído, o jogo encerra imediatamente. A pontuação é o número de clientes atendidos até aquele momento.

Vitória. Se todos os clientes forem atendidos sem que nenhum perca a paciência, o jogador vence. A pontuação é o número total de clientes atendidos.

Os clientes aguardam atendimento em uma **fila (FIFO)** (ver §4).

3.4. Ações do jogador

As seguintes ações consomem um turno cada:

- Preparar um ingrediente
- Adicionar um ingrediente ao hambúrguer

- Remover o ingrediente do topo do hambúrguer

Ações de consulta (ver estoque, ver pedido, etc.) não consomem turno.

4. Estruturas de dados

Esta seção descreve o papel e os requisitos de cada estrutura de dados utilizada no trabalho. O código de todas as estruturas foi dado em aula e deve ser adaptado pelo grupo. A tabela abaixo apresenta uma visão geral; os detalhes estão nas subseções a seguir.

Papel no jogo	**Estrutura(s)**
Armazenamento e busca de pedidos	AVL
Histórico de partidas	Lista contígua
Cache de buscas recentes	Tabela hash
Receita e montagem do pedido	Lista encadeada + Pilha
Fila de atendimento	Fila (FIFO)
Estoque de ingredientes perecíveis	Heap mínimo
Ordenação do histórico	Um algoritmo (à escolha)
Medição de tempo	`timer.h` / `timer.c`

4.1. AVL — armazenamento dos pedidos

Cada pedido é identificado por um código numérico único e representado em duas linhas no arquivo: a primeira é o código; a segunda é uma string de 5 a 7 caracteres com a receita (Tabela 1).

▶ Os pedidos devem ser carregados em uma AVL na inicialização do programa, antes de exibir o menu, a partir do arquivo recebido via linha de comando (ver Apêndice C). O código foi dado em aula. O formato do arquivo está descrito em §6.

4.2. Lista contígua — histórico de partidas

▶ O histórico lido de `historico.txt` deve ser carregado em uma lista contígua alocada dinamicamente na inicialização do programa, antes de exibir o menu, a partir do arquivo recebido via linha de comando (ver Apêndice C). Cada entrada contém o nome do jogador e a pontuação obtida.

A escolha da lista contígua é intencional: os algoritmos de ordenação das aulas operam sobre ela e permitem acesso por índice em $O(1)$, o que é necessário para implementações eficientes de *quicksort* e *mergesort*. Ao encerrar uma partida, a nova entrada é **acrescentada ao final da lista e salva no arquivo**.

Integridade dos dados. A ordenação utilizada para exibição ou para as medições de desempenho **não deve modificar** o arquivo `historico.txt`. Ao gravar uma nova entrada, todas as partidas anteriores devem ser preservadas no arquivo.

4.3. Tabela hash — cache de pedidos

▶ O programa deve manter uma tabela hash, indexada pelo código do pedido, como cache das buscas realizadas durante a sessão. O código foi dado em aula. A política de tratamento de colisões recomendada é encadeamento separado (cada posição contém

uma lista encadeada das entradas que mapearam para aquele índice); o grupo pode adotar outra política e implementá-la.

A lógica de acesso é: ao sortear um código para o novo cliente, *consulta-se primeiro a tabela hash*; se o pedido já estiver no cache (*hit*), ele é retornado sem consultar a AVL; se não estiver (*miss*), busca-se na AVL e o resultado é inserido na hash.

Convenções para as métricas da hash:

- **Colisão:** ocorre quando uma inserção encontra a posição já ocupada por ao menos uma entrada. Conta-se uma colisão por inserção que gera conflito, independentemente do comprimento da lista naquele índice.
- **Fator de carga:** $\lambda = n/m$, onde n é o número de entradas ocupadas e m é o tamanho da tabela.
- **Redimensionamento:** não é obrigatório e não faz parte dos requisitos deste trabalho.

4.4. Lista encadeada — receita de cada pedido

▶ **A receita de cada pedido deve ser armazenada como uma lista encadeada, onde cada nó representa um ingrediente na ordem de montagem (de baixo para cima).** Esta representação permite percorrer os ingredientes em ordem e verificar o hambúrguer montado pelo jogador. Use o código das aulas.

4.5. Pilha — montagem do hambúrguer

▶ **A montagem do hambúrguer deve ser gerenciada por uma pilha (LIFO): adicionar um ingrediente equivale a empilhá-lo; remover o ingrediente do topo equivale a desempilhá-lo.** Só deverá ser aceito o hambúrguer cuja pilha, lida de baixo para cima, corresponda exatamente à lista encadeada do pedido. Use o código das aulas.

O pão de baixo já começa no prato (não faz parte da receita). Não deve ser possível adicionar ingredientes por cima do pão de cima.

Verificação da receita. Como a interface padrão da pilha não permite percorrê-la de baixo para cima sem desempilhar, o grupo pode: (a) adaptar o TAD para adicionar uma operação de travessia, ou (b) usar uma estrutura auxiliar temporária (cópia, vetor, etc.) para a comparação, preservando a pilha original ao final. Qualquer abordagem é válida desde que a pilha não seja destruída numa consulta.

4.6. Fila — clientes

▶ **Os clientes devem ser gerenciados por uma fila (FIFO): clientes entram no final e são atendidos na ordem de chegada.** Use o código das aulas.

4.7. Heap mínimo — ingredientes perecíveis

Ingredientes são divididos em três categorias:

- **Sem preparo, quantidade infinita:** ketchup, mostarda, alface, picles, pão.
- **Com preparo, perenes:** queijo suíço, queijo cheddar, cebola, tomate. Cada ação de preparo produz uma unidade no estoque. Uma vez preparados, aguardam indefinidamente.
- **Com preparo, perecíveis:** carne, ovo frito, bacon. Uma vez preparados, devem ser descartados após um número de turnos *definido pelo grupo* (pode ser diferente por ingrediente).

▶ **O estoque de cada ingrediente perecível deve ser implementado como um heap mínimo (um por tipo: carne, ovo frito, bacon), com prioridade no turno de validade — o ingrediente mais próximo de vencer fica sempre no topo.** Use o código das aulas.

Regras de validade:

- O turno de vencimento de uma unidade é: **turno de preparo + validade configurada**.
- **No início de cada turno,** antes de qualquer ação do jogador, o programa verifica o topo de cada heap perecível e descarta todas as unidades com turno de vencimento $\leq$ turno atual, repetindo até o heap estar vazio ou o topo ser válido.
- Ao **consumir** um perecível, retira-se a unidade do topo do heap correspondente (a que vence primeiro). Se o heap estiver vazio, a ação não pode ser realizada.

Caso um ingrediente já usado no hambúrguer seja removido pelo jogador, ele é descartado e não volta ao estoque.

A Tabela 1 resume todos os ingredientes.

Ingrediente	**Símbolo**	**Requer preparo?**	**Perecível?**
Pão (de cima)	z	Não	Não
Ketchup	k	Não	Não
Mostarda	m	Não	Não
Picles	p	Não	Não
Alface	a	Não	Não
Queijo suíço	s	Sim	Não
Queijo cheddar	q	Sim	Não
Cebola	c	Sim	Não
Tomate	t	Sim	Não
Carne	n	Sim	Sim
Ovo frito	o	Sim	Sim
Bacon	b	Sim	Sim

Tabela 1: Características e representação de cada ingrediente.

5. Medições obrigatórias e Relatório de Desempenho

O programa coleta métricas ao longo de toda a execução e as consolida em dois momentos: **imediatamente após o carregamento** dos pedidos e ao **encerrar a partida**, quando é exibido automaticamente um **Relatório de Desempenho** completo — tanto na tela quanto gravado em arquivo.

Arquivo de log obrigatório. Todas as medições devem ser gravadas em `medicoes.log`, criado (ou sobrescrito) a cada execução. O arquivo deve registrar cada métrica com seu nome e valor, de forma legível, para consulta após o encerramento do programa. A ausência do arquivo ou a omissão de qualquer métrica implica desconto no critério *Medições obrigatórias*.

5.1. Após o carregamento: AVL e histórico

Logo após o carregamento inicial, registrar as métricas abaixo no log.

AVL:

- Número de nós inseridos
- Altura da árvore resultante
- Número de rotações (inserção)
- Tempo total de carregamento

Histórico (lista contígua) — contexto:

- Número de entradas carregadas

Convenções:

- **Altura:** árvore vazia = 0; árvore com apenas a raiz = 1.
- **Rotações:** uma rotação dupla conta como **duas** rotações simples.
- **Comparação (busca):** cada comparação entre a chave procurada e a chave de um nó visitado conta como uma comparação.
- **Média de comparações por busca:** se não houver nenhuma busca na partida, reportar 0.

Durante a partida, acumule silenciosamente (sem exibir a cada turno) o **tempo total** e o **número de comparações** das buscas na AVL. Esses dados serão exibidos apenas no Relatório de Desempenho.

5.2. Relatório de Desempenho (ao encerrar a partida)

Ao encerrar a partida — seja por vitória, derrota ou saída voluntária — o programa exibe automaticamente o Relatório de Desempenho e o grava no log antes de retornar ao menu. O relatório deve conter:

1. **AVL — buscas durante a partida:**
   - Total de comparações e média por busca;
   - Tempo total acumulado de buscas.
2. **Tabela hash — estatísticas de cache:**
   - Total de *hits* e *misses*; taxa de acerto (%);
   - Fator de carga ($\lambda = n/m$);
   - Total de colisões durante inserções.
3. **Ordenação do histórico** (executada quando o usuário acessa a opção **Histórico** no menu):
   - O algoritmo ordena uma **cópia independente** do conjunto de dados — a cópia fica fora do intervalo cronometrado;
   - Tempo de execução do algoritmo escolhido;
   - Em acessos subsequentes ao Histórico, a medição é repetida e o log atualizado.

5.3. Como medir o tempo

Os arquivos `timer.h` e `timer.c`, disponibilizados junto com os demais códigos das aulas, encapsulam a medição de tempo. O código completo das funções está no Apêndice A. Basta incluí-los no projeto e usar a API abaixo.

API do timer (C99+)

```
#include "timer.h"

Timer t;
timer_iniciar(&t);

/* ... operacao a medir ... */

double s = timer_parar(&t);   /* tempo em segundos */
printf("Tempo: %.6f s\n", s);
```

Acumule os tempos de buscas individuais sem exibir a cada turno; exiba total e média apenas no Relatório de Desempenho e grave no `medicoes.log`.

6. Arquivos de teste

O programa recebe os nomes dos arquivos via linha de comando (`argv[1]` e `argv[2]`), portanto **não deve assumir nomes fixos**. Serão disponibilizadas duas versões de cada arquivo:

Pedidos (código numérico + receita)

- **Versão de desenvolvimento** (`pedidos.txt`): 837.253 pedidos, disponibilizada nas aulas. Use para testar e depurar o programa.
- **Versão final** (`pedidos_final.txt`): divulgada **no dia da apresentação**. O programa deverá executá-la **sem nenhuma modificação de código**, recebendo o nome via `argv[1]`. Os pedidos podem conter repetição de códigos entre as duas versões.

Cada pedido está estruturado em duas linhas: a primeira é um inteiro (código identificador único do pedido) e a segunda é a string da receita.

Histórico (nome + pontuação)

- **Versão de desenvolvimento** (`historico.txt`): grande volume de entradas, disponibilizada nas aulas. Use para testar e medir o desempenho da ordenação.
- **Versão final** (`historico_final.txt`): divulgada **no dia da apresentação**. O programa deverá ordená-la **sem nenhuma modificação de código**, recebendo o nome via `argv[2]`.

Cada registro do histórico está estruturado em duas linhas: a primeira é uma string (de no máximo 20 caracteres) com o nome do jogador, e a segunda é um inteiro representando a pontuação. Não existe garantia de nenhum destes valores ser único.

O programa não deve assumir o tamanho dos arquivos. As quantidades devem ser lidas dinamicamente.

7. Interface

O jogo pode ser realizado inteiramente em texto, via terminal. Devem estar visíveis: o pedido do cliente (ingredientes exigidos), o hambúrguer montado até o momento, a quantidade de cada ingrediente no estoque e os turnos restantes.

8. Extras

Funcionalidades extras podem ser implementadas desde que **não comprometam nem substituam** nenhum requisito obrigatório, valendo até **1 ponto** extra (saturando em 10). Sugestões:

- **Interface gráfica** (fortemente sugerida): vale diretamente **1,0 ponto extra** se implementada;
- Tempo de cozimento (em turnos) para ingredientes preparáveis;
- Seletor de dificuldade;
- Poderes de uso único (estender paciência do cliente, multiplicar ingredientes, etc.);
- **Splay Tree:** implementar também uma Splay Tree para armazenar os pedidos e incluí-la na comparação com a AVL (altura, comparações, tempo de carga). Comentar no relatório o que a análise amortizada prevê e o que foi observado.

9. Entrega

Entregar no Moodle um único arquivo `.zip` (`tp-grupoX.zip`, onde **X** é o número do grupo disponibilizado no material da disciplina) contendo:

- **Código-fonte:** arquivos `.c`/`.h` (ou `.cpp`/`.hpp`) organizados por módulo (`main.c`, `avl.c`, `hash.c`, `heap.c`, `pilha.c`, `fila.c`, `lista.c`, `timer.c`, etc.). O programa deve compilar e rodar com o comando indicado no Apêndice C.
- **Arquivo de log:** `medicoes.log` gerado pela execução com `pedidos.txt` e `historico.txt` (versões de desenvolvimento), contendo todas as medições obrigatórias (§5).
- **Relatório em PDF** (máximo 8 páginas, incluindo capa), seguindo o template disponibilizado nas aulas. O relatório deverá conter obrigatoriamente as seguintes seções, nesta ordem:
  1. **Introdução:** apresentação do grupo e visão geral do trabalho. Indicar quais requisitos obrigatórios foram implementados e quais não foram, com justificativa para os ausentes.
  2. **O jogo:** descrever o jogo implementado — como funciona a partida, quais ações estão disponíveis, quais parâmetros configuráveis foram adotados (paciência, número de clientes, validade dos perecíveis, etc.) e a justificativa das escolhas. Incluir também funcionalidades extras implementadas, se houver.
  3. **Estruturas de dados:** para cada estrutura utilizada no jogo (AVL, lista contígua, tabela hash, lista encadeada, pilha, fila e heap), descrever brevemente sua finalidade e seu papel na implementação. Detalhar e justificar as adaptações realizadas no código fornecido. Para a tabela hash, especificar a função hash, o tamanho da tabela e a política de colisão adotados, com justificativa.

4. **Medições e análise:** apresentar os resultados das medições obrigatórias definidas na Seção 5, organizados em tabelas. Para cada medição, comparar os resultados obtidos com os valores teóricos esperados e discutir eventuais discrepâncias.
5. **Conclusão:** descrever as principais dificuldades encontradas, a divisão de tarefas entre os integrantes e os principais aprendizados obtidos durante o desenvolvimento do projeto.
6. **Declaração de uso de IA** (quando aplicável): informar as ferramentas de inteligência artificial utilizadas, suas respectivas finalidades e os artefatos gerados com seu auxílio. Caso nenhuma ferramenta tenha sido utilizada, esta seção poderá ser omitida.

10. Avaliação

A avaliação do trabalho será realizada com base nos seguintes critérios:

- **Funcionamento da aplicação (50%):** atendimento aos requisitos obrigatórios especificados (estruturas de dados, mecânicas do jogo, medições e geração do log).
- **Organização e documentação do código (10%):** legibilidade, modularização, organização e documentação da implementação.
- **Relatório (30%):** descrição da solução desenvolvida, justificativa das estruturas de dados utilizadas e análise da complexidade das principais operações (ver §9).
- **Apresentação (10%):** clareza na exposição do trabalho, domínio técnico da solução implementada e capacidade de responder a questionamentos sobre o projeto.

Plágio. Plágio de código entre grupos ou de fontes externas não referenciadas implica nota zero.

Uso de inteligência artificial. É **permitido** utilizar ferramentas de IA generativa (ChatGPT, Claude, Copilot, ou similares) para tarefas auxiliares que *não* implementem nem alterem os algoritmos e as estruturas de dados avaliados. Exemplos de usos permitidos: leitura e escrita de arquivos, tratamento de erros de entrada e saída, manipulação de argumentos da linha de comando, formatação de menus e relatórios, e utilização das funções de medição de tempo fornecidas pelo professor.

É **proibido** utilizar essas ferramentas para gerar, modificar, explicar por meio de sugestões de código ou depurar implementações das estruturas de dados, dos algoritmos de ordenação, das operações sobre essas estruturas ou da lógica de contagem das métricas avaliadas. Isso inclui AVL, tabela hash, listas, pilha, fila, heap, algoritmo de ordenação e métricas como comparações, rotações e colisões.

Em caso de dúvida sobre se determinado uso é permitido, o grupo deverá **consultar o professor antes** de utilizá-lo. **Todo uso de IA, sem exceção, deve ser declarado no relatório**, indicando a ferramenta utilizada, a finalidade e os artefatos produzidos com seu auxílio. A omissão de uso de IA é tratada como plágio.

11. Datas importantes

- **Definição do grupo no Moodle:** data disponível no Moodle.
- **Entrega do trabalho/Apresentação:** data disponível no Moodle.
- **Dia da apresentação:** divulgação de `pedidos_final.txt` e `historico_final.txt` e execução ao vivo com esses arquivos via linha de comando.

A. Medição de tempo: referência prática

Os arquivos abaixo são distribuídos junto com o material da disciplina. Basta copiá-los para o projeto e incluí-los na compilação (ver Apêndice C).

Versão C

timer.h

```
#ifndef TIMER_H
#define TIMER_H
#include <time.h>
typedef struct { clock_t inicio; } Timer;
void   timer_iniciar(Timer *t);
double timer_parar(Timer *t);   /* retorna tempo em segundos */
#endif
```

timer.c

```
#include "timer.h"
void timer_iniciar(Timer *t) { t->inicio = clock(); }
double timer_parar(Timer *t) {
    return (double)(clock() - t->inicio) / CLOCKS_PER_SEC;
}
```

Versão C++

timer.hpp

```
#ifndef TIMER_HPP
#define TIMER_HPP
#include <chrono>
struct Timer { std::chrono::steady_clock::time_point inicio; };
void   timer_iniciar(Timer *t);
double timer_parar(Timer *t);   /* retorna tempo em segundos */
#endif
```

timer.cpp

```
#include "timer.hpp"
void timer_iniciar(Timer *t) {
    t->inicio = std::chrono::steady_clock::now();
}
double timer_parar(Timer *t) {
    auto fim = std::chrono::steady_clock::now();
    return std::chrono::duration<double>(fim - t->inicio).count();
}
```

B. Exemplo de partida comentada

Configuração do exemplo

- **Pedidos na AVL:** 1001 → `nkmpz`, 1042 → `qampkz`, 2017 → `nbomkz`.
- **Validade dos perecíveis:** carne = 3 turnos; ovo frito = 2 turnos; bacon = 3 turnos.
- **Clientes:** 2, enfileirados no início. Fila inicial: [C1, C2].

Pré-jogo: carregamento

Os pedidos são lidos de `argv[1]` e inseridos na AVL; o histórico é lido de `argv[2]` e carregado na lista contígua (neste exemplo, 15 entradas). O programa grava no `medicoes.log`:

Métrica	**AVL**
Nós inseridos	3
Altura resultante	2
Rotações (inserção)	1
Tempo de carregamento	0,000012 s
Histórico — contexto	
Entradas carregadas	15

A tabela hash começa vazia. Os heaps dos perecíveis começam vazios.

Cliente 1 — pedido 1001: `nkmpz`

Início do atendimento. C1 está na frente da fila. O código 1001 é sorteado.

- **Hash: MISS** (tabela vazia) → busca na AVL: 2 comparações → receita `nkmpz` encontrada.
- Pedido inserido na hash: {1001 → `nkmpz`}.

Os turnos seguintes montam o hambúrguer `nkmpz` (preparar carne + adicionar n, k, m, p, z). Ao fechar com o pão de cima, a pilha é comparada com a lista encadeada do pedido: ✓. **C1 atendido.**

C2 passa à frente. O código 1001 é sorteado novamente:

- **Hash: HIT** → receita `nkmpz` retornada sem consultar a AVL.

C2 é atendido da mesma forma. Fila vazia: **Vitória!**

Caso: perecível não usado a tempo

Com validade de 3 turnos, o jogador prepara três carnes seguidas:

- **T1:** prepara carne → validade = turno 4. Heap: [4]
- **T2:** prepara outra → validade = turno 5. Heap: [4, 5]
- **T3:** prepara mais uma → validade = turno 6. Heap: [4, 5, 6]
- **T4 (início):** topo = 4 = turno atual → descarte automático. Heap: [5, 6]
- **T4 (ação):** usa carne → remove topo (5). Heap: [6]

Relatório de Desempenho

Ao encerrar a partida, o programa exibe e grava em `medicoes.log`. Ao acessar o menu **Histórico**, os tempos de ordenação são acrescentados:

```
=== RELATORIO DE DESEMPENHO ===

-- AVL: carregamento --
Nos inseridos:                   3
Altura resultante:               2
Rotacoes (insercao):             1
Tempo de carregamento:           0.000012 s

-- Historico: carregamento --
Entradas carregadas:             15

-- AVL: buscas durante a partida --
Total de buscas:                 1
Total de comparacoes:            2
Media de comparacoes por busca:  2.00
Tempo total acumulado:           0.000005 s

-- Tabela hash --
Tamanho (m):                 101
Entradas ocupadas (n):       1
Fator de carga (lambda):     0.01
Total de hits:               1
Total de misses:             1
Taxa de acerto:              50.0%
Total de colisoes:           0

-- Ordenacao do historico (ao acessar Historico) --
Tamanho: 15 entradas
Merge Sort:  0.000003 s

================================
```

C. Ponto de entrada e compilação

▶ O programa deve receber os arquivos de entrada via linha de comando usando `argc` e `argv`.

Como funciona `argc`/`argv`. Quando o usuário executa o programa passando argumentos, o sistema operacional os entrega ao `main` como um vetor de strings:

```c
int main(int argc, char *argv[]) {
    /* argc  = numero de argumentos (incluindo o nome do
       executavel) */
    /* argv[0] = nome do executavel
                                          */
    /* argv[1] = primeiro argumento  (pedidos.txt)
                          */
    /* argv[2] = segundo argumento   (historico.txt)
                        */
    if (argc < 3) {
        fprintf(stderr, "Uso: %s <pedidos> <historico>\n", argv
           [0]);
        return 1;
    }
    FILE *fp = fopen(argv[1], "r");  /* abre o arquivo de pedidos
        */
    FILE *fh = fopen(argv[2], "r");  /* abre o arquivo de
       historico */
    /* ... */
}
```

Compilação e execução:

C (gcc)

```
gcc -o dado_burger main.c avl.c hash.c heap.c pilha.c \
    fila.c lista.c timer.c -lm -Wall -std=c99

./dado_burger pedidos.txt historico.txt          # desenvolvimento
./dado_burger pedidos_final.txt historico_final.txt  #
   apresentacao
```

C++ (g++)

```
g++ -o dado_burger main.cpp avl.cpp hash.cpp heap.cpp pilha.cpp \
    fila.cpp lista.cpp timer.cpp -Wall -std=c++17

./dado_burger pedidos.txt historico.txt          # desenvolvimento
./dado_burger pedidos_final.txt historico_final.txt  #
   apresentacao
```

Os arquivos devem estar no mesmo diretório do executável. Na primeira execução, `historico.txt` pode não existir — o programa deve tratar esse caso abrindo com histórico vazio.